

Data Structures: Foundations

Minseok Jeon
DGIST

November 2, 2025

Contents

1. Introduction
2. Choosing a Programming Language
3. Variables, Control Flow, and Loops
4. Functions and Recursion
5. Arrays and Strings Fundamentals
6. Memory Model: Stack vs Heap
7. Debugging and Testing
8. Using Online Judges
9. Summary

Introduction

Why Foundations Matter

Course Philosophy

Build a strong programming foundation **before** diving into complex data structures

What You'll Master:

- Programming language fundamentals
- Control flow and recursion
- Memory models (stack vs heap)
- Debugging and testing skills
- Problem-solving patterns

Why This Matters:

- Data structures require solid coding skills
- Understanding memory is crucial
- Testing prevents subtle bugs
- Good habits accelerate learning

Key Principle

Implement every structure **from scratch** at least once to truly understand it

Choosing a Programming Language

Language Comparison

Language	Strengths	Best For	Learning Curve
C/C++	Maximum control, performance, STL	Competitive programming, systems	Steep (pointers, memory)
Java	Strong libraries, OOP, widely used	Interviews, enterprise	Moderate
Python	Readable, fast to write	Learning, prototyping	Easy

For Beginners

Python or Java

- Focus on concepts, not syntax
- Less memory management

For Performance/CP

C++

- Learn STL containers
- Manual memory control

Language-Specific Considerations

C++ Example

```
1 #include <vector>
2 #include <iostream>
3
4 int main() {
5     std::vector<int> arr = {1, 2, 3};
6     arr.push_back(4); // Dynamic resize
7
8     // Manual memory for complex types
9     int* ptr = new int[100];
10    delete[] ptr; // Must free!
11
12    return 0;
13 }
```

Manual memory, explicit types

Python Example

```
1 # Dynamic typing, GC
2 arr = [1, 2, 3]
3 arr.append(4) # Auto-resize
4
5 # No manual memory management
6 # Objects cleaned up automatically
7
8 # But watch mutability!
9 list1 = [1, 2, 3]
10 list2 = list1 # Same reference
11 list2.append(4)
12 print(list1) # [1, 2, 3, 4]
```

Automatic memory, dynamic types

Important

Data structures behave differently: manual memory in C/C++ vs garbage collection in Java/Python

Variables, Control Flow, and Loops

Core Concepts

Data Types & Scope

- Basic types: int, float, bool
- Type conversion and casting
- Variable scope and lifetime

Control Flow

```
1 # If/else with early returns
2 def process(x):
3     if x < 0:
4         return "negative"
5     elif x == 0:
6         return "zero"
7     else:
8         return "positive"
```

Loop Patterns

- Index-based vs iterator-based
- Nested loops: $O(n^2)$, $O(n^3)$
- Break/continue usage
- Loop invariants

Common Pitfall

```
1 # Off-by-one error!
2 for i in range(len(arr) - 1): # Missing last!
3     print(arr[i])
4
5 # Correct:
6 for i in range(len(arr)):
7     print(arr[i])
```

Nested Loops and Complexity

Time Complexity Examples

```
1 # O(n) - Single loop
2 for i in range(n):
3     print(i)
4
5 # O(n^2) - Nested loop
6 for i in range(n):
7     for j in range(n):
8         print(i, j)
9
10 # O(n^3) - Triple nested
11 for i in range(n):
12     for j in range(n):
13         for k in range(n):
14             print(i, j, k)
```

Functions and Recursion

Function Fundamentals

Key Concepts

- Pass by value vs reference
- Return values and side effects
- Function purity
- Single responsibility principle

Pure Function

```
1 # Pure: no side effects
2 def add(a, b):
3     return a + b
4
5 # Impure: modifies state
6 def append_item(lst, item):
7     lst.append(item) # Side effect!
```

Best Practices

- Small, focused functions
- Clear, descriptive names
- Document preconditions
- Handle edge cases

C++ Reference

```
1 // Pass by value (copy)
2 void func1(vector<int> v) {
3     v.push_back(1); // Original unchanged
4 }
5
6 // Pass by reference (no copy)
7 void func2(vector<int>& v) {
8     v.push_back(1); // Original modified
9 }
```

Recursion Basics

Essential Components

- **Base case:** Stopping condition (prevents infinite recursion)
- **Recursive case:** Progress toward base case
- **Stack frames:** Each call adds to call stack

Factorial

```
1 def factorial(n):
2     # Base case
3     if n <= 1:
4         return 1
5     # Recursive case
6     return n * factorial(n - 1)
7
8 # Call stack for factorial(3):
9 # factorial(3) -> 3 * factorial(2)
10 # factorial(2) -> 2 * factorial(1)
11 # factorial(1) -> 1 (base case)
12 # Unwind: 2 * 1 = 2, then 3 * 2 = 6
```

Binary Search

```
1 def binary_search(arr, target, left, right):
2     # Base case: not found
3     if left > right:
4         return -1
5
6     mid = left + (right - left) // 2
7
8     # Base case: found
9     if arr[mid] == target:
10        return mid
11
12    # Recursive cases
13    if arr[mid] > target:
14        return binary_search(arr, target,
15                               left, mid - 1)
```

Recursion Patterns and Pitfalls

Common Patterns

- **Divide and Conquer:**
Merge sort, quick sort
- **Tree Traversals:**
In-order, pre-order, post-order
- **Backtracking:**
Permutations, N-Queens
- **Dynamic Programming:**
Fibonacci with memoization

Common Pitfalls

- Missing base case
→ Infinite recursion
- Incorrect base case
→ Wrong results
- Deep recursion
→ Stack overflow
- No progress to base
→ Infinite loop

Solution for Deep Recursion

Convert to iterative with explicit stack

Arrays and Strings Fundamentals

Arrays: Core Properties

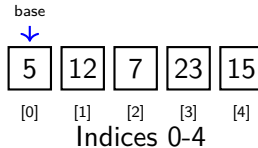
Key Characteristics

- Contiguous memory
- $O(1)$ indexing
- Fixed size vs resizable
- Cache-friendly

Common Operations

Operation	Time
Access	$O(1)$
Search	$O(n)$
Insert (end)	$O(1)$ amortized
Insert (middle)	$O(n)$
Delete (middle)	$O(n)$

Array Structure



Memory Address

$\text{addr}[i] = \text{base} + i \times \text{size}$

Strings: Special Arrays

String Properties

- Arrays of characters
- Immutable (Java/Python) vs Mutable (C char arrays)
- Concatenation costs
- Substring operations

C++ (Mutable)

```
1 char s[] = "hello";
2 s[0] = 'H'; // OK: "Hello"
3
4 // C++ string class
5 string str = "hello";
6 str += " world"; // Efficient
7 str[0] = 'H'; // OK
```

Python (Immutable)

```
1 s = "hello"
2 # Can't modify: s[0] = 'H' # Error!
3
4 # Concatenation creates new string
5 s = s + " world" # O(n)
6
7 # Better for multiple ops:
8 parts = []
9 parts.append("hello")
10 parts.append(" world")
```

Common Tasks

- Substring search
- Pattern matching
- Palindrome checking
- Reversal

• Character frequency

Memory Model: Stack vs Heap

Stack Memory

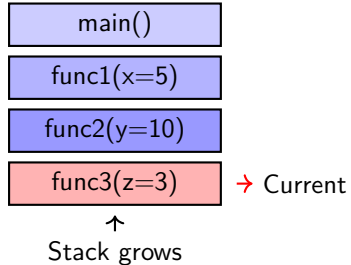
Characteristics

- Stores function call frames
- Parameters and local variables
- Fast allocation/deallocation
- Limited size (typically 1-8 MB)
- Automatic cleanup
- LIFO (Last In, First Out)

Stack Overflow

Deep recursion can exhaust stack space
→ Convert to iteration or increase stack size

Call Stack



Heap Memory

Characteristics

- Dynamic memory allocation
- Objects with longer lifetimes
- Larger capacity (GBs)
- Slower than stack
- Manual (C/C++) or GC (Java/Python)

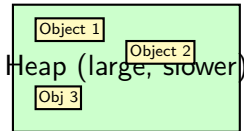
Memory Management

C/C++: Manual
malloc/free, new/delete

Java/Python: Automatic
Garbage collection

Memory Layout

Stack (small, fast)



Pointers and References

C++ Pointers

```
1 int x = 10;
2 int* ptr = &x; // Pointer to x
3
4 *ptr = 20;      // Modify via pointer
5 cout << x;     // 20
6
7 // Heap allocation
8 int* arr = new int[100];
9 arr[0] = 5;
10 delete[] arr;  // Must free!
11
12 // Dangling pointer (BAD!)
13 int* p = new int(42);
14 delete p;
15 cout << *p;   // Undefined!
```

Python References

```
1 # Objects are references
2 list1 = [1, 2, 3]
3 list2 = list1      # Same reference!
4
5 list2.append(4)
6 print(list1)      # [1, 2, 3, 4]
7
8 # To copy:
9 list3 = list1.copy() # Shallow copy
10 list3.append(5)
11 print(list1)      # [1, 2, 3, 4]
12
13 # Deep copy for nested structures
14 import copy
15 nested = [[1, 2], [3, 4]]
16 deep = copy.deepcopy(nested)
```

Practice

C/C++: Allocate/free arrays, avoid leaks and dangling pointers

Java/Python: Understand when objects are shared and mutated

Debugging and Testing

Debugging Strategies

Essential Tools

- **Debugger:** Breakpoints, step through, watch variables
- **Assertions:** Capture invariants early
- **Logging:** Strategic print statements
- **Binary search:** Isolate bug location

Don't Just Print!

Use a proper debugger:

- Set breakpoints
- Inspect variable state
- Step line by line
- Understand execution flow

Debugging Process

1. **Reproduce:** Minimal test case
2. **Isolate:** Which function fails?
3. **Inspect:** Variable values at failure
4. **Hypothesize:** What could cause this?
5. **Test:** Verify hypothesis
6. **Fix:** Apply solution
7. **Validate:** Ensure fix works

Tools by Language

C/C++: gdb, lldb, valgrind

Java: JUnit, debugger

Python: pdb, unittest, pytest

Testing Best Practices

Testing Principles

- Start with unit tests
- Test edge cases
- Small and large inputs
- Property-based thinking
- Measure performance

Edge Cases to Test

- Empty input
- Single element
- Duplicates
- Negative numbers
- Boundary values

Python Unit Test

```
1 import unittest
2
3 class TestArrayOps(unittest.TestCase):
4     def test_reverse_normal(self):
5         arr = [1, 2, 3, 4]
6         reverse(arr)
7         self.assertEqual(arr, [4, 3, 2, 1])
8
9     def test_reverse_empty(self):
10        arr = []
11        reverse(arr)
12        self.assertEqual(arr, [])
13
14    def test_reverse_single(self):
15        arr = [1]
16        reverse(arr)
17        self.assertEqual(arr, [1])
18
19    def test_reverse_property(self):
20        arr = [1, 2, 3]
21        reverse(reverse(arr))
22        self.assertEqual(arr, [1, 2, 3])
23
24 if __name__ == '__main__':
25     unittest.main()
```


Using Online Judges

Online Judge Platforms

Popular Platforms

- **LeetCode**: Interview prep
- **HackerRank**: Competitions, hiring
- **Codeforces**: Competitive programming
- **AtCoder**: Japanese CP platform
- **TopCoder**: Algorithms, marathons

Benefits

- Immediate feedback
- Test against edge cases
- Compare solutions
- Track progress

Problem-Solving Approach

1. **Read** carefully, note constraints
2. **Design** algorithm with complexity
3. **Implement** cleanly
4. **Test** edge cases manually
5. **Submit** and iterate
6. **Reflect** on solution
7. **Document** patterns learned

Avoid Pitfalls

- Don't just copy solutions
- Re-implement after understanding
- Pay attention to constraints

Common Problem Patterns

Pattern	Description	Example Problems
Two Pointers	Use two indices moving through data	Palindrome, pair sum
Sliding Window	Fixed/variable size window	Max subarray sum
Fast & Slow Pointers	Detect cycles, find middle	Linked list cycle
Stack	LIFO for matching/parsing	Valid parentheses
BFS/DFS	Graph/tree traversal	Level-order, paths
Binary Search	Divide search space	Search sorted array

Summary

Key Takeaways

Language and Tools

- Choose one language and master it (Python/Java for learning, C++ for performance)
- Understand memory management for your language
- Learn debugging tools and use them effectively

Core Programming Skills

- Master control flow, loops, and avoid off-by-one errors
- Understand recursion: base case, recursive case, stack frames
- Know array and string fundamentals, including complexity

Memory Model

- Stack: fast, limited, automatic (function frames)
- Heap: larger, slower, manual or GC (dynamic objects)

Next Steps

Moving Forward

With these foundations in place, you're ready to:

- Study linear data structures (arrays, linked lists, stacks, queues)
- Implement each structure from scratch
- Analyze time and space complexity
- Apply structures to real-world problems
- Build toward more complex structures (trees, graphs, hash tables)

Recommended Practice

1. Implement basic array operations (reverse, rotate, search)
2. Write recursive solutions for factorial, Fibonacci, binary search
3. Practice two-pointer and sliding window problems

4. Solve 10-20 easy problems on [LeetCode](#) or [HackerRank](#)

Thank You!

Questions?

*“The best way to learn data structures
is to implement them yourself.”*